

台灣房價預測系統

30 分鐘口說版 | 資料工程 · GIS · LightGBM · Django 部署

970,806 筆

訓練資料

LightGBM

模型

Django + MySQL

線上系統

Cloudflare

部署

重點：不是只做模型，而是把實價登錄、外部房源、AI 推論與網站部署串成完整資料產品。

專案問題與目標

把分散的房源資訊轉成可解釋、可部署的估價服務

- 房價資訊分散：實價登錄、591、信義房屋欄位格式不同 • 單看開價難以判斷合理性，需要歷史成交、地理位置與生活機能一起評估
- 目標：建立可搜尋、可收藏、可即時估價的房價預測平台

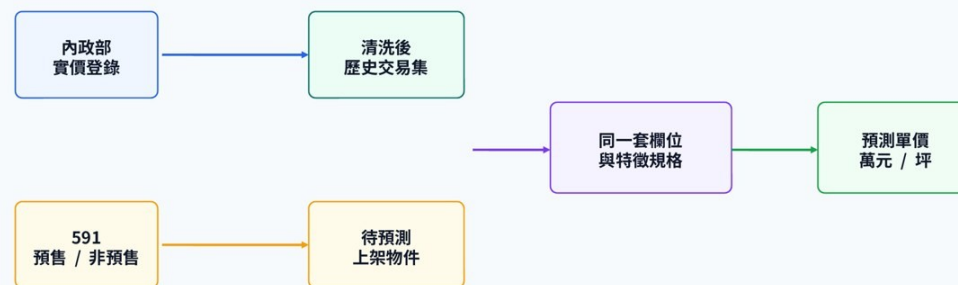
輸入：地址、坪數、屋齡、樓層

輸出：預測單價、預測總價、行情比較

02

資料來源：實價登錄建立訓練集，591 提供待預測物件

先建立可信的歷史市場資料，再把外部平台物件轉成相同特徵格式。



資料抽取策略

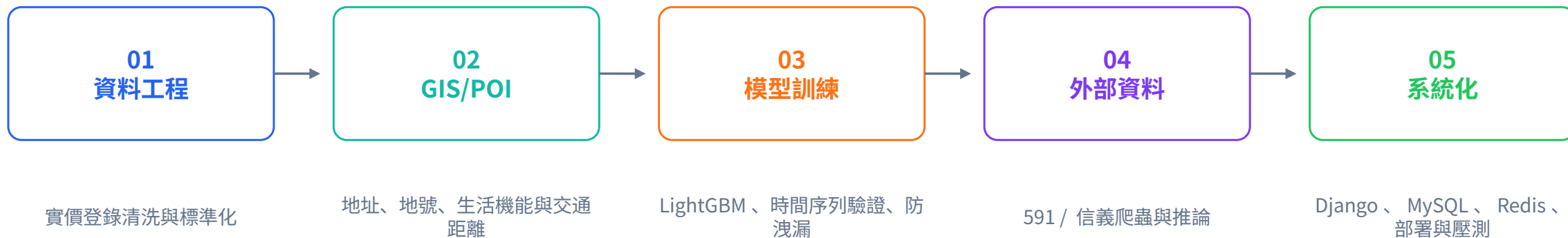
- 實價登錄：依年月 / 縣市批次讀取 CSV，兼容 utf-8-sig 與 cp950
- 591：Session + CSRF + list/detail 頁補欄位，折成預售與非預售處理
- 所有新資料都要轉成與訓練資料一致的欄位與單位

Portfolio deck - Real Estate Price Prediction

早期簡報中的資料來源與推論流程

30 分鐘口試主線

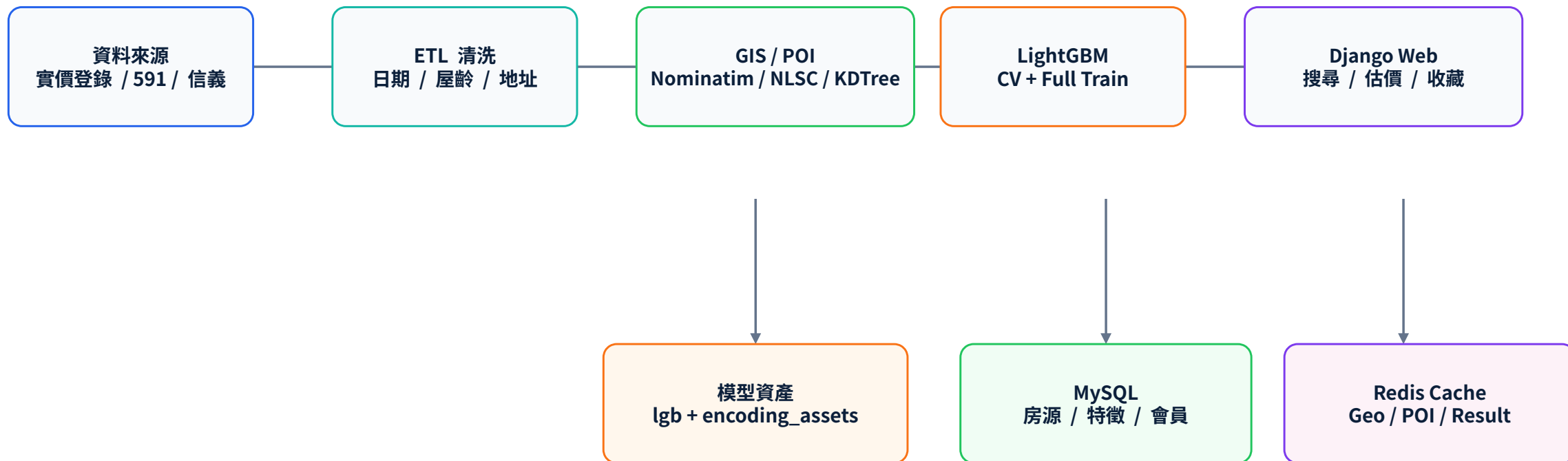
從資料到模型，再到系統上線



口試節奏：資料工程約 8 分鐘、模型約 6 分鐘、外部資料約 5 分鐘、系統部署約 7 分鐘、Demo 與結論約 4 分鐘。

整體系統總架構

新增整合圖：資料來源、模型資產與網站服務如何串接



這張圖是六份 PDF 的統整：從訓練資料到線上系統，最後由 Cloudflare 對外提供服務。

PART 1

資料工程與基礎清洗

從實價登錄 CSV 到可訓練的標準資料表

資料來源與原始資料挑戰

內政部實價登錄開放資料是模型訓練主資料

- 讀取 F:\桃竹苗\final\house 下多個 a_lvr_land_a 類型 CSV • 原始欄位超過 30 欄，包含地址、日期、建物型態、面積、總價與車位資訊
- 挑戰：民國日期、字串型數值、車位價格干擾、特殊交易備註與住宅用途混雜

908,853 筆

第一階段整合後

20 欄

核心欄位

第一階段：批次清洗與合併

21(1).py：將多年度多縣市 CSV 清洗成統一格式



交易時間與屋齡

將 1050520 / 981201 等民國日期解析為西元年、月，並以成交年 - 完工年計算屋齡。

車位去噪

若車位有總價與面積，先扣除車位價格與車位面積，再換算住宅淨單價。

特殊交易排除

透過備註關鍵字移除親友交易、信託、拍賣、瑕疵屋、地上權、僅車位等干擾樣本。

這一步的成果：把原始交易 CSV 轉成「欄位一致、單價可比較、住宅用途為主、可進一步建模」的合併資料集。

STEP 1 / 資料工程與前處理 • 04

第一階段：批次清洗與合併

第一階段清洗與合併結果

將多年度、多縣市 CSV 轉成統一格式

- 補齊縣市與行政區，建立「完整行政區」欄位 • 交易年月日轉成成交年、成交月與交易距今月數
- 坪數、單價、主建物、樓層等欄位轉為可計算格式
- 保留預售 / 非預售標記與屋齡缺失標記，支援後續模型特徵

產出：標準化後 CSV / PKL，作為 Kaggle 特徵工程與模型訓練的輸入。

第一階段輸出：合併後訓練基礎資料

修正後統計：908,853 筆、20 欄，作為 Kaggle 前處理輸入

908,853

合併後樣本數

20

輸出欄位數

138.7 MB

記憶體使用量

地理

縣市、鄉鎮市區、完整行政區、土地位置建物門牌

時間

成交年、成交月、交易距今月數

建物

屋齡、建物型態、坪數、移轉層次、總樓層數、電梯

目標 / 標記

單價_萬元坪、is_presale、age_was_missing

模型前資料表結構

類型	欄位數	代表欄位
float64	7	成交年、坪數、單價
int64	3	交易距今月數、標記
str	10	縣市、建物型態、地址

這個資料集尚未直接訓練；下一步會在 Kaggle 進行更細的欄位正規化、缺失補值、地理特徵與價格邊界處理。

輸入 Kaggle 前處理

STEP 1 / 資料工程與前處理 · 05

合併後訓練基礎資料

訓練前特徵工程總覽

由交易資料擴充到地理、生活機能與交通特徵

- 新增 $\text{time_index} = \text{成交年} * 12 + \text{成交月}$ ，讓模型理解時間趨勢
- 經緯度：一般地址用 Nominatim；地號資料用國土測繪中心 API
- POI：超商、銀行、餐飲、醫療、停車場、宗教場所
- 交通：高鐵、台鐵、捷運最近距離

970,806 筆

最終資料

38 欄

模型特徵

訓練前最終資料集

19

合併基礎清洗資料、經緯度、POI 與交通特徵後，形成模型訓練輸入

970,806

最終樣本數

38

最終欄位數

281.5 MB

記憶體使用量

25 float

主要數值特徵

特徵群	代表欄位
基礎交易	縣市、鄉鎮市區、完整行政區、成交年、成交月、交易距今月數
建物屬性	屋齡、建物型態、坪數、移轉層次、總樓層數、電梯、主建物面積
價格目標	單價_萬元坪
座標	唯一地理位置、time_index、緯度、經度
POI 計數	超商、銀行、餐飲、醫療、停車場 300m / 500m 數量
距離特徵	最近宮廟、高鐵、台鐵、捷運距離
後續訓練階段	area_last_month_price、總編碼等訓練時再加入

Step 1-2 / 資料工程、地理編碼與空間特徵工程

Interview Portfolio

訓練前最終資料集

PART 2

GIS 與空間特徵工程

地址、地號、POI 與交通距離是模型差異化核心

地址清理：建立「乾淨地理位置」

降低地址噪音，提升地理編碼成功率

- 移除縣市、行政區、樓層、鄰、交叉路口與備註類文字
- 統一台 / 臺、全形 / 半形數字與特殊符號
- 多門牌只保留主門牌，避免 Geocoder 被錯誤地址干擾
- 結果用於產生唯一地理位置與經緯度查詢

口說範例：台北市松山區敦化北路二段 10 樓之 2 → 敦化北路二段

地址清理：建立「乾淨地理位置」

04

先把土地位置建物門牌整理成可查詢經緯度的地址字串

1 縣市 / 行政區移除

從土地位置建物門牌中移除縣市與鄉鎮市區，避免重複字串干擾查詢。

2 字元標準化

市→市、移除空白、全形數字轉半形、長破折號轉成半形、。

3 門牌壓縮

2-6-7 號、11 之 5 之 3-1 號等多段門牌壓縮成主要門牌。

4 移除非定位資訊

刪除鄰、交叉口、附近、旁邊、對面、基地、門牌尚未申請等模糊描述。

5 手動修正常見異常

針對苗 128 線、嘉 64、龍岡道旁等常見地名寫法做 replace 修正。

6 刪除樓層與棟戶

移除 A5-03、10 樓、地下 2 樓、八樓之二等與地理定位無關的資訊。

核心轉換

土地位置建物門牌 → 乾淨地理位置
例：台北市松山區敦化北路二段 10 樓之 2 → 敦化北路二段

Step 1-2 / 資料工程、地理編碼與空間特徵工程

Interview Portfolio

地址清理：建立乾淨地理位置

Nominatim 地理編碼與 12 層降級策略

一般地址使用本地 OSM / Nominatim 服務取得經緯度

- 本地 Docker 建立 Nominatim，避免外部 API 成本與速率限制
- query_api 先比對「縣市 + 鄉鎮市區」，再降級到只比對縣市
- fetch_lat_lon 依序嘗試 full address、乾淨地址、去號、去弄、去巷等版本
- aiohttp + semaphore 控制併發，提升速度但避免打爆服務

12 層地址降級策略 + 非同步查詢

08

用 aiohttp + Semaphore 批次查詢唯一地址，並把結果精準回填到原資料

標記	查詢地址版本	目的
1	縣市 + 區 + 乾淨地理位置	最精準完整地址
2	只有乾淨地理位置	避免縣市重複造成查詢失敗
3-4	乾淨地址 full / only	退回到主要門牌
5-6	去號 full / only	退回到路段或巷弄
7-8	去弄 full / only	去除更細地址
9-10	去巷 full / only	退回到路段級
11-12	最後地址 full / only	保底候選地址

批次查詢優化

先用 drop_duplicates 只查唯一地址，避免 90 萬筆重複打 API。
CONCURRENT_LIMIT=60 控制同時請求量，兼顧速度與本機服務負載。

結果回填

用 results_map 儲存 key → (lat, lon, use_type)。
原始資料再動態合成相同 key，精準回填「緯度、經度、用哪個地址」。

可追蹤性

「用哪個地址」記錄命中層級，後續可檢查座標品質：數字越小代表越精準。

Step 1-2 / 資料工程 - 地理編碼與空間特徵工程

Interview Portfolio

12 層地址降級策略與非同步查詢

地號與小段資料：走地籍代碼流程

一般地址與地號資料分流處理，補齊更多經緯度

- 包含「地號 / 小段」的地址無法直接用 Nominatim 查準 • 先用段名 .csv 建立 lookup：縣市、鄉鎮、段、小段、所區碼、代碼
- parse_land 抽出地號與地號 8 碼，再呼叫國土測繪中心 API
- 多執行緒查詢搭配 sleep / retry，避免限流或解析失敗

國土測繪中心座標查詢：地號資料補經緯度

13

將 office、sect、landno 組成查詢參數，取得地籍中心座標

查詢資料準備

段名代碼表欄位重新整理：所區碼→ office、代碼→ sect。
有小段與無小段分開 merge，避免小段空值造成對不到。
landno 使用 8 碼格式。

API 查詢

呼叫
landmaps.nslc.gov.tw/S_Maps/qryTileMapIndex
參數包含 type、flag、office、sect、landno、callback
回傳 JSONP，需用正則抽出括號內 JSON 再解析。

回填策略

多執行緒完成後用 index 對齊回填緯度與經度。
只覆蓋原本缺失、解析失敗或查無資料的列。
正常已有座標的資料不動。

地籍座標查詢概念

office=AD, sect=0600, landno=07690000 → 回傳 cy / cx → 緯度 / 經度

Step 1-2 / 資料工程、地理編碼與空間特徵工程

Interview Portfolio

國土測繪中心座標查詢

OSM POI 與交通站點抽取

從 taiwan.osm.pbf 建立生活機能與交通點位資料

- POI：shop、amenity 等 OSM 標籤，分類為超商、銀行、餐飲、醫療、停車場、宗教場所
- 交通：railway=station 搭配 station/operator/highspeed 標籤分類 THSR / TRA / MRT
- 將點與面統一取 centroid，輸出 lat / lon / poi_type

OSM POI 抽取：建立生活機能點位資料

從 taiwan-260512.osm.pbf 抽出全台灣商店、金融、醫療、餐飲、停車場與宗教場所

資料來源

使用 pyosm 載入台灣 OSM PBF。
不只讀點資料，也處理 Way/Polygon。
所有幾何統一取 centroid 作為經緯度。

自訂篩選器

shop=True：零售店與便利商店。
amenity 包含 restaurant、cafe、fast_food、clinic、hospital、pharmacy、bank、parking、place_of_worship 等。
涵蓋正面生活機能與可能負面設施。

輸出資料

osm_id、name、poi_type、lat、lon。
shop 類標記為 shop_xxx。
amenity 類標記為 amenity_xxx。
輸出 taiwan_comprehensive_pois.csv / pkl。

Step 1-2 / 資料工程、地理編碼與空間特徵工程

Interview Portfolio

OSM POI 抽取

交通站點抽取：高鐵 / 台鐵 / 捷運分類

從 OSM railway=station 中精準分出不同交通系統

車站資料抽取

使用 railway=station 過濾全台軌道車站。
額外抓 station、operator、highspeed 屬性。
點與面資料同樣取 centroid。

分類邏輯

THSR：名稱含高鐵 / THSR、operator 為台灣高速鐵路，或 highspeed=yes。
MRT：station=subway/light_rail，或名稱 / operator 含捷運、地鐵、輕軌。
其餘歸類為 TRA_station。

輸出用途

欄位：osm_id、name、station_type、lat、lon。
輸出 taiwan_stations_t2.pkl。
後續用來計算最近高鐵、台鐵、捷運距離。

Step 1-2 / 資料工程、地理編碼與空間特徵工程

Interview Portfolio

交通站點抽取

交通特徵分類
station_type ∈ {THSR_station, TRA_station, MRT_station}

KDTree 空間特徵： 90 萬筆房屋快速計算周邊機能

將 OSM 點位轉為模型可使用的空間特徵

- 以 KDTree 建立不同類別設施的空間索引
- 計算 300m / 500m 內的超商、餐飲、銀行、醫療、停車場數量
- 計算最近宗教場所、高鐵、台鐵、捷運站距離
- 避免房屋 × POI 暴力比對，讓 90 萬筆資料可在合理時間完成

KDTree 空間特徵： 90 萬筆房屋快速計算周邊機能

18

將 POI 與車站建立空間索引，避免逐筆暴力距離計算

特徵	計算方式	意義
poi_convenience_count_300m / 500m	超商 KDTree 半徑查詢	生活便利程度
poi_bank_count_500m	銀行 KDTree 半徑查詢	商業活動指標
poi_food_count_300m / 500m	餐飲 POI 半徑查詢	生活圈熱度
poi_medical_count_500m	診所 / 醫院 / 藥局半徑查詢	醫療可近性
poi_parking_count_500m	停車場半徑查詢	開車便利性
distance_to_nearest_worship_m	最近宗教場所距離	潛在嫌惡 / 文化設施距離
distance_to_thsr/tra/mrt_m	最近車站 KDTree k=1	交通可達性

技術細節

Step 1-2 / 資料工程、地理編碼與空間特徵工程

在台灣緯度下以 1 度約
Interview Portfolio

KDTree 空間特徵計算

PART 3

模型訓練與防資料洩漏

LightGBM + 時間序列驗證 + 行情特徵

訓練資料漏斗：先保證資料品質

價格與面積篩選後保留約 99% 資料

- 原始資料 970,806 筆，經價格與面積條件過濾極端值
- 最終訓練保留 961,726 筆，保留比例 99.06%
- 過濾目的不是大量刪資料，而是避免極端錯誤紀錄污染模型
- 目標欄位使用 $\log_{1p}$ (單價 _ 萬元坪)，使誤差更穩定

99.06%

保留比例

8.15%

Full Train MAPE

房價預測系統

專業技術簡報 | 資料工程・特徵工程・模型訓練・部署

簡報目標與面試展示重點

99.06%

訓練資料保留率

8.15%

全訓練 MAPE

2.38 萬

全訓練 MAE

38+ 欄

完整特徵規模

本版整合內容

- 已確認你貼的是 Full-Train 全量訓練程式
- 補上 geo_last_month_price、geo_txn_count、geo_price_ratio 三個地理歷史特徵
- 詳細拆解 area_last_month_price、merge_asof、Expanding Encoding 與資料洩漏防護
- 同時保留前面資料清洗、地理編碼、POI/KDTree 特徵工程內容

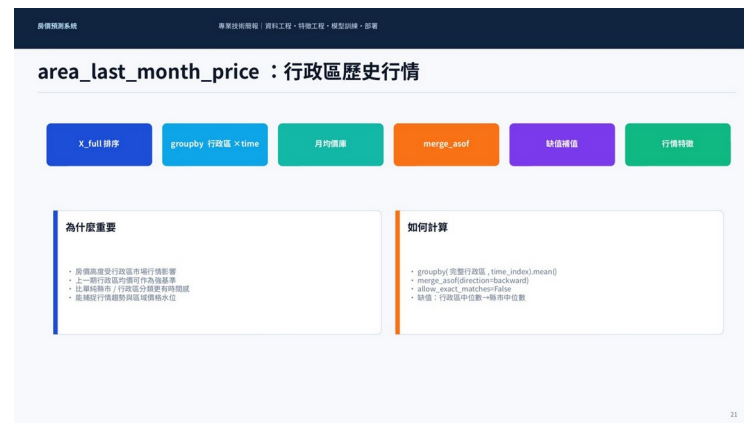
02

訓練資料漏斗與最終指標

area_last_month_price：行政區歷史行情特徵

使用 merge_asof 嚴格限制只能看過去資料

- 依「完整行政區 + time_index」建立每月區域平均單價
- merge_asof direction=backward，預測某月時只允許使用更早月份行情
- 若行政區缺歷史資料，依序回退到行政區中位數、縣市中位數或全域值
- 好處：模型可學到區域行情水位，而不是只靠地址文字或經緯度



area_last_month_price



merge_asof 防洩漏

時間安全分層編碼：縣市 → 行政區 → 地理位置

Target Encoding 也要避免未來資料洩漏

- `compute_expanding_stats`：同一群組只用當前列以前的累積平均
- 低樣本量地理位置用行政區均價做平滑，避免少量樣本過擬合
- 預售屋與成屋分流：預售用行政區行情，成屋用地理位置行情
- 輸出總 _ 編碼，並在推論時用 `encoding_assets` 保持一致邏輯



LightGBM + TimeSeries 5-Fold 驗證

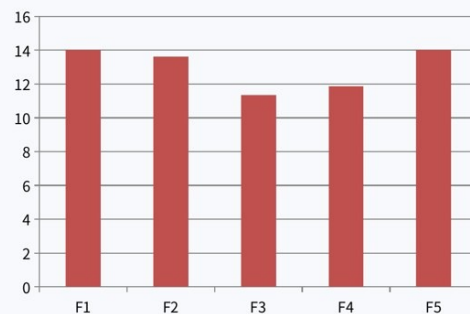
用時間順序切分，避免隨機切分造成過度樂觀

- 模型：LightGBM regression_l1，目標是單價 (萬元 / 坪)
- 特徵包含坪數、屋齡、樓層、電梯、經緯度、POI、交通距離、行情特徵
- 每個 Fold 內重新計算歷史行情與分層編碼，避免驗證集洩漏到訓練特徵
- Early stopping 找到合理迭代輪數，再用全資料進行 Full Train

房價預測系統

專業技術簡報 | 資料工程 · 特徵工程 · 模型訓練 · 部署

TimeSeries 5-Fold CV 結果



驗證表現

- Fold1 : MAPE 14.01% , MAE 4.2924 萬
- Fold2 : MAPE 13.62% , MAE 4.1407 萬
- Fold3 : MAPE 11.34% , MAE 3.8217 萬
- Fold4 : MAPE 11.88% , MAE 4.6052 萬
- Fold5 : MAPE 14.00% , MAE 4.8334 萬

29

TimeSeries 5-Fold CV 結果

Full Train：產出生產環境模型資產

用全部有效資料重訓，並保存推論所需對照表

- 使用 CV 得到的迭代輪數，對全資料進行最終訓練 • 保存 lgb_full_train_final.txt 作為生產模型
- 保存 encoding_assets.pkl：歷史行情、地理行情與推論用資產
- 保存 feature_dtypes / label_encoders，確保線上推論欄位型別一致

2.38 萬 / 坪

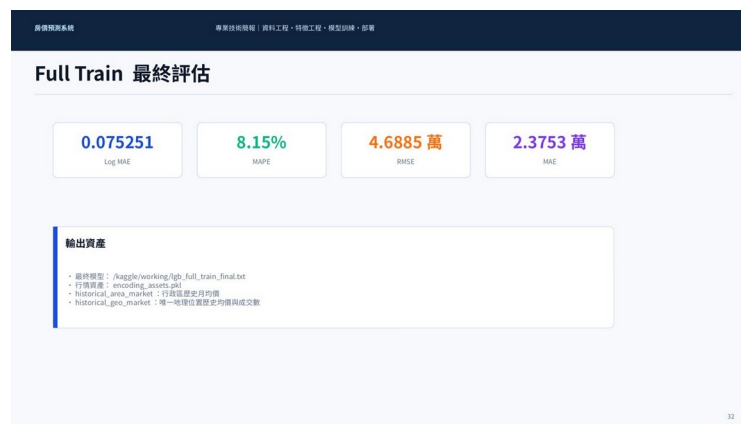
Full Train MAE

4.69 萬 / 坪

Full Train RMSE



Full-Train 完整流程



Full Train 最終評估

模型成果：訓練集與分資料來源評估

補充指標：591 預售與 591 非預售訓練表現

2.87%

591 預售訓練

MAE 1.10 萬 / 坪

11.78%

591 非預售訓練

MAE 3.25 萬 / 坪

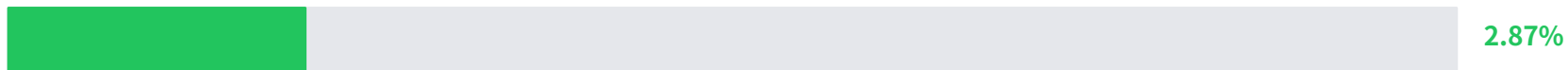
8.15%

Full Train

MAE 2.38 萬 / 坪

MAPE 比較

591 預售訓練



2.87%

591 非預售訓練



11.78%

Full Train



8.15%

口說時要分清楚：訓練集評估代表模型學習能力；外部平台評估因為是「開價」而非「成交價」，誤差會提高。

PART 4

外部資料抓取與推論

591、信義房屋資料如何進入模型流程

外部資料整合流程

591 與信義房屋作為線上房源推論與驗證資料

- 591 預售：newhouse API 抓列表，再進 detail 頁補總樓層、交屋時間、建設公司與座標
- 591 非預售：BFF API 抓屋齡、坪數、價格、樓層、圖片與社區資訊
- 信義房屋：解析 Next.js 的 __NEXT_DATA__ 取得房源列表
- 最後全部轉成模型訓練時相同的 Schema 再推論



591 預售： List API + Detail Page 雙階段抓取

列表拿基本欄位，Detail 補足模型關鍵特徵

- 先呼叫 search API，依 rid / page 抓 hid、區域、建案、坪數、價格區間。再抓 housing-detail HTML，從 JSON-LD 取經緯度、樓層規劃、預計交屋、建設公司
- 無法從 JSON-LD 取得時，使用 HTML fallback 補欄位
- 使用 Session / Cookie / CSRF 初始化，降低請求失敗率



591 預售資料抓取與欄位整理



parse_detail：補足 detail 頁欄位

爬蟲防封鎖與信義房屋資料解析

以速率控制與 checkpoint 提升穩定性

- 591：multiprocessing Pool 控制併發，random sleep 降低封鎖風險 • detail HTML 批次下載到暫存資料夾，再同步到雲端硬碟，避免中途失敗資料遺失
- 信義：safe_get 控制所有請求最小間隔，遇到 429 隨機等待 30-90 秒後重試
- 解析 __NEXT_DATA__ / buyReducer，抽出 house_list 並定期 checkpoint



外部房源前處理：對齊訓練時的模型欄位

把平台欄位轉成 LightGBM 可吃的 Feature View

- 縣市、行政區、建物型態與地址格式統一：台 / 臺、宜蘭縣 / 宜蘭市等
- 交屋時間與屋齡 2 共同推導 is_presale、屋齡與 age_was_missing
- 樓層、坪數、屋齡常是區間或多值，先轉成數字候選清單
- explode 成多組可能組合後推論，最後回聚合為預測最小值與最大值

這頁已重新排版，圖固定在右側，不再遮到說明文字。



推論特徵：行情、編碼與型別保持一致

線上推論必須與訓練流程完全一致

- `get_area_price_feature`：從 `historical_area_market` 找過去月份區域行情，缺值逐層回退
- `get_prediction_encoded_values`：依建案、地理位置、行政區、縣市做分層回退
- 載入 `model_dtypes / label_encoders`，確保欄位型別與訓練時相同
- 預測 `log` 空間後用 `expm1` 還原成萬元 / 坪



歷史行情與編碼特徵



外部資料推論結果與價格帶分析

外部資料評估：模型在真實平台開價上的表現

外部平台多為開價，不等同於實際成交價



信義預售 15.95%

最佳外部組

信義非預售 27.26%

最高誤差組

開價 $\neq$ 成交價

主要原因

PART 5

Django 系統與部署

把模型接到網站、資料庫與外網服務

Django 後端系統架構

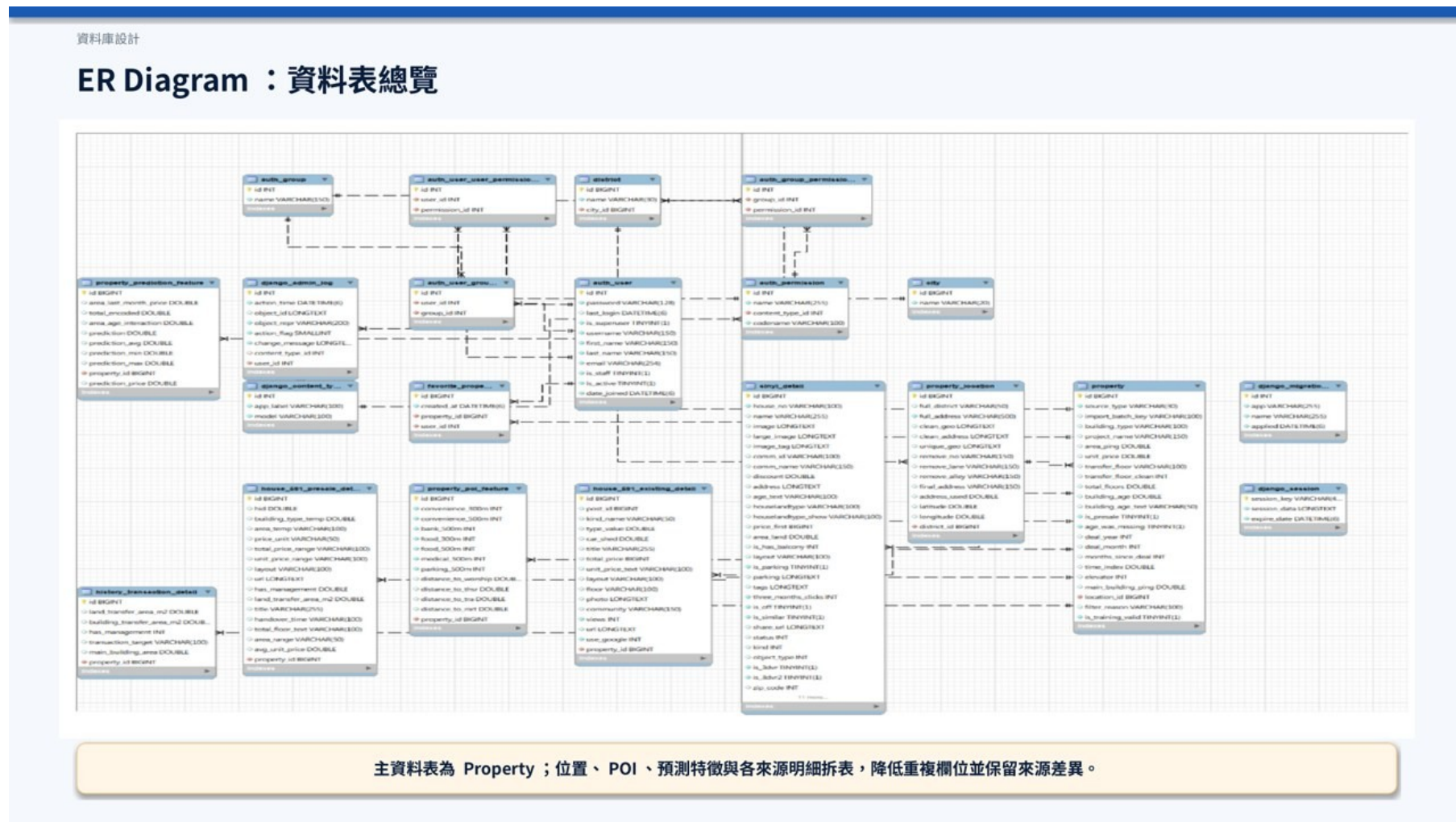
網站不是 Notebook，而是完整的 Web 應用

- Django models 對應 MySQL 資料表，管理房源、位置、POI、預測特徵與會員收藏
- views.py 負責搜尋、詳情頁、登入註冊、收藏與 AI 預測邏輯
- ml 模組拆成 address_utils、geocode_utils、poi_utils、encoding_utils、feature_builder、predictor
- LightGBM 模型與 encoding_assets 作為生產環境推論資產



資料庫 ER Diagram

保留 v6 的完整資料庫關聯圖



房價預測系統 | 30 分鐘口說版 v8

MySQL 匯入流程： PKL / CSV 到 ORM 資料表

批次寫入降低 SQL 次數，提高匯入效率

- import_pkl2.py 讀取清洗後 PKL / CSV，建立 City、District、Property 等物件。先去重與建立索引，避免重複城市、行政區與房源資料
- PropertyLocation、PropertyPOIFeature、PredictionFeature 拆表寫入
- 使用 bulk_create 批次寫入，大幅降低 ORM 單筆 insert 成本



AI 預測資料流：從表單到結果頁

feature_builder 是線上估價速度與準確度的核心

- ai_predict_form.html 收集城市、行政區、地址、坪數、屋齡、樓層等資料
- feature_builder 產生與訓練一致的欄位：經緯度、POI、行情、編碼與交互特徵
- predictor 載入 LightGBM 模型，輸出預測單價並回推總價
- ai_predict_result.html 顯示預測結果、行情比較與資料來源資訊



Cache 優化：把最慢的外部計算變成可重用結果

Redis / Django Cache 提升線上估價速度

- geo cache：同一地址不重複呼叫 Nominatim
- poi cache：同一經緯度不重複計算 KDTree POI 特徵
- area cache：同一行政區不重複查歷史行情
- encode cache：同一地區與建物型態不重複計算分層編碼
- result cache：完全相同輸入一小時內直接回傳結果

效果：首次預測需要跑完整特徵流程；命中快取後可接近即時回應。

Cache 優化

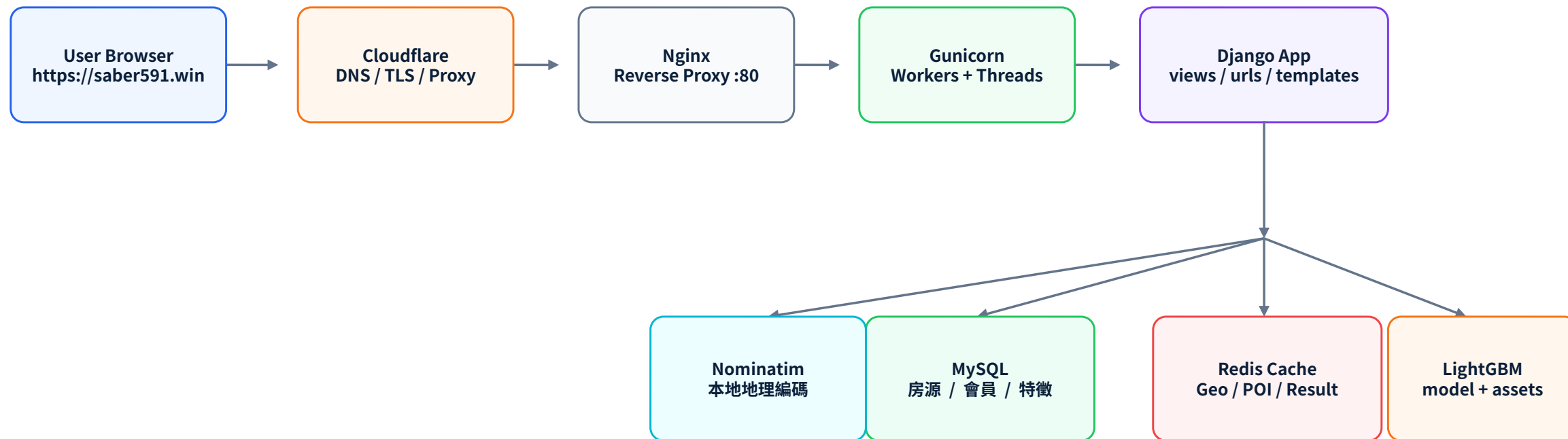
ml/feature_builder.py：內部快取設計

- geo cache：key 由 city/district/clean_info JSON + MD5 生成，TTL 30 天；避免同地址重複打 Nominatim。
- poi cache：key 由 round(lat/lon, 6) 生成，TTL 30 天；避免同座標重複做 KDTree 查詢。
- area_price cache：key=city/district/year/month，TTL 1 天；行政區月行情每天算一次即可。
- encode cache：key 包含 city/district/building_type/unique_geo/project/is_presale，TTL 1 天；避免反覆查 encoding assets。
- 這些子快取讓 AI 結果快取失效時，仍能快速完成特徵工程。

Feature Builder 內部快取設計

正式部署架構圖

新增重畫版：線條直、層級清楚、避免原圖歪斜



部署口說重點：Cloudflare 處理網域與 TLS，Nginx 反向代理到 Gunicorn；Django 串接 MySQL、Redis、LightGBM 與本地 Nominatim。

壓力測試： 100 人同時載入的服務能力

ApacheBench 驗證本地與 Cloudflare 端效能



結論：本地端 Nginx + Gunicorn 在 /property/ 可支援 100 concurrency、703 req/s ； Cloudflare 端受 TLS 與網路延遲影響，但仍穩定。

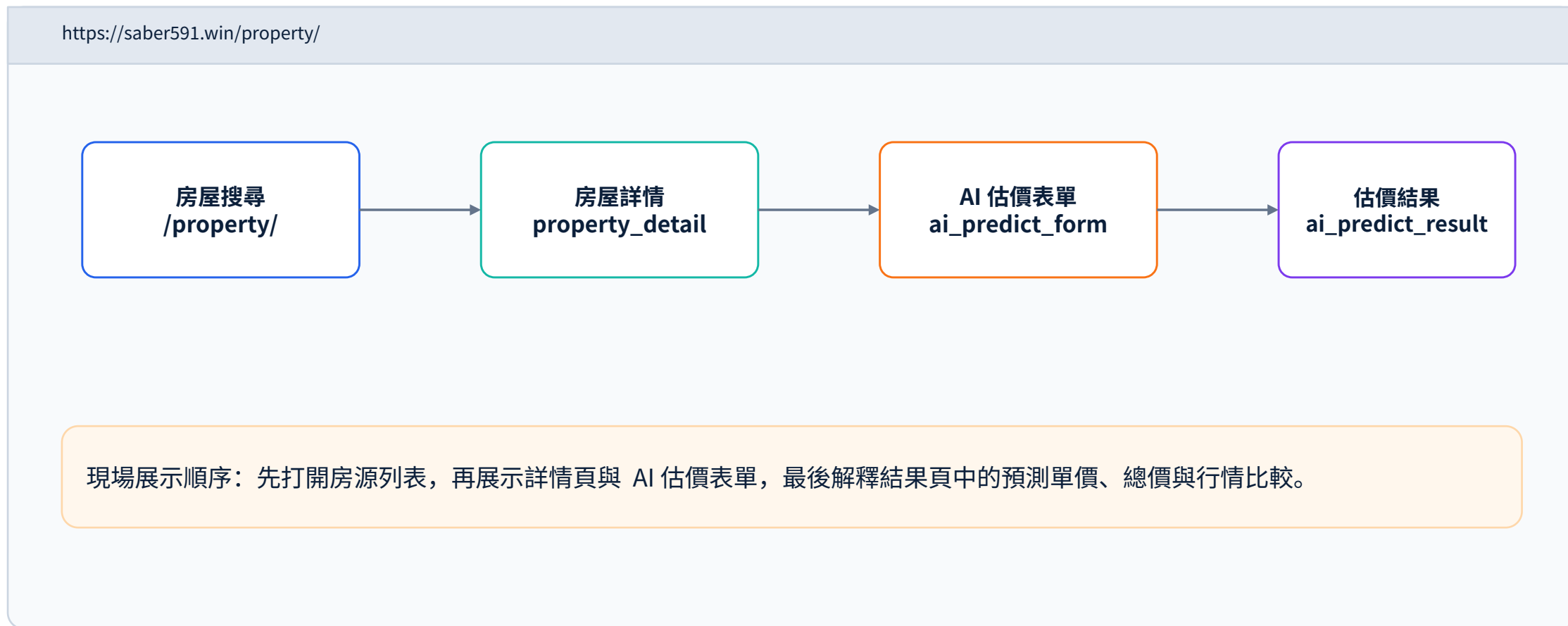
ApacheBench 測試結果

測試項目	本地端	Cloudflare
/property	703 req/s	40 req/s
/login	3171 req/s	198 req/s
/register	3044 req/s	198 req/s

原始 AB 測試表

Live Demo：現場展示 saber591.win/property

用線上頁面取代靜態 Case Study



結論：完成一個可運作的房價資料產品

資料工程

970,806 筆資料、38 個特徵

GIS / POI

Nominatim + NLSC + OSM/KDTree

模型訓練

LightGBM + TimeSeries CV + Full Train

系統落地

Django + MySQL + Redis + Cloudflare

最大價值：把真實房地產資料從蒐集、清洗、特徵工程、模型訓練到線上 AI 估價完整落地。